

# 第1章 初识Vue.js

# 学习目标/Target

---

了解前端技术的发展，能够说出使用框架开发项目的优势

了解什么是Vue，能够说出Vue的基本概念

了解Vue的特性，能够说出Vue的4个特性

了解Vue的版本，能够说出Vue 2和Vue 3的区别

# 学习目标/Target

---

掌握Visual Studio Code编辑器的使用方法，能够使用Visual Studio Code编辑器进行项目开发

掌握Node.js环境的搭建，能够独立完成Node.js的下载和安装

掌握常见的包管理工具的使用方法，能够应用npm和yarn相关命令下载、升级、卸载包

掌握Vite的使用方法，能够使用Vite创建Vue 3项目

## 章节概述/ Summary

---

在前端开发中，一个优秀的框架可以帮助用户解决一些常见的问题，有助于高效地完成工作。Vue.js（简称Vue）作为前端开发常用的框架之一，不仅可以提高项目的开发效率，而且可以改善开发体验。为了帮助读者对Vue有一个初步的认识，本章将对Vue的基础知识进行详细讲解。

# 目录/Contents

---

1.1

前端技术的发展

1.2

Vue简介

1.3

Vue开发环境

1.4

使用Vite创建Vue 3项目

**1.1**

# 前端技术的发展

## 1.1 前端技术的发展

---



- 了解前端技术的发展，能够说出使用框架开发项目的优势

## 1.1 前端技术的发展

---



前端开发的基础语言为HTML、CSS和JavaScript, 其中, HTML用于搭建页面的内容结构; CSS用于美化页面的显示效果; JavaScript用于处理用户和页面之间的交互行为。

## 1.1 前端技术的发展

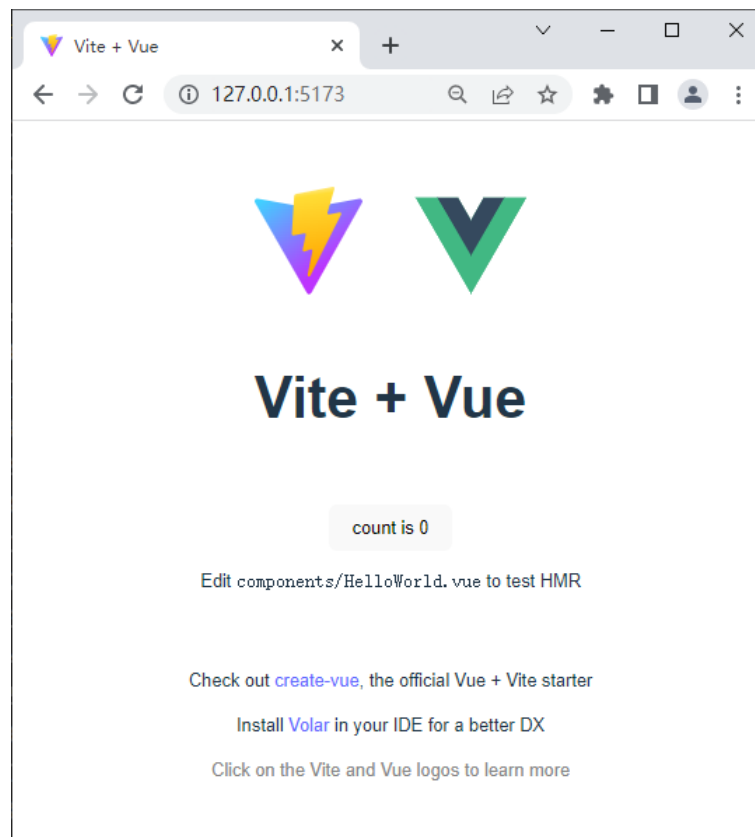
---

为了简化DOM操作和减少开发过程中的浏览器兼容性问题，jQuery提供了一个选择器引擎，它比其他引擎查询速度更快，并为不同浏览器之间的JavaScript不兼容提供了隐式处理方法，因此jQuery深受开发人员的欢迎。jQuery的核心思想是使开发人员仅编写少量的代码就能实现更多的功能。



## 1.1 前端技术的发展

为了使移动端网页的使用体验更接近用户习惯，移动端网页通常会做成单页Web应用（Single Page Web Application）的形式。单页Web应用在使用过程中只需要加载一个HTML页面，而传统的网页是用户每单击一个链接都需要加载相应的HTML页面。



## 1.1 前端技术的发展

---

为了提高开发效率，市面上出现了基于MVVM模式的前端开发框架，例如Angular、React、Vue等。这些框架以数据为核心，使用户关注业务逻辑的处理，减少了手动的DOM操作。在Angular、React和Vue这3个框架中，Vue体积较小，在使用上更容易上手、更加灵活。



## 1.1 前端技术的发展

---



### 多学一招：什么是单页Web应用

单页Web应用将所有的功能局限于一个Web页面中，仅在该页面的初始化时加载相应的资源（必要的HTML、CSS和JavaScript代码）。

在页面加载完成后，所有的操作都在这个页面上完成，且不会因用户的操作而进行页面的重新加载或跳转，而是利用JavaScript动态地变换页面的内容，从而实现页面与用户的交互。

## 1.1 前端技术的发展

---



### 多学一招：什么是单页应用

单页Web应用有以下3个**优点**。

- ① **良好的交互体验**。在单页Web应用中，内容的改变不需要重新加载整个页面，响应速度更快。
- ② **良好的前后端分离开发模式**。后端专注于提供**API**，更容易实现API的复用。
- ③ **减轻服务器的压力**。单页Web应用中的数据是通过Ajax获取的，不需要重新加载，服务器的压力较小。

## 1.1 前端技术的发展

---



### 多学一招：什么是单页Web应用

任何一种技术都有局限性，对于单页Web应用来说，主要的**问题**有以下2个。

- ① **首屏加载慢**，在首次加载时需要将页面中所用到的资源全部加载。
- ② **不利于搜索引擎优化**（Search Engine Optimization, SEO）。对于单页Web应用，搜索引擎请求到的HTML页面可能不是包含所有数据的最终渲染页面，这样就很不利于内容被搜索引擎搜索到。

## 1.1 前端技术的发展

---



多学一招：什么是单页Web应用

随着技术的进步，上述问题已经有了相应的解决方案。

对于第1个问题，可以通过路由[懒加载](#)、[代码压缩](#)、[网络传输压缩](#)等方式解决；对于第2个问题，可以通过[服务器端渲染](#)（Server-Side Rendering, SSR）技术解决。

1.2

## Vue简介

## 1.2.1 什么是Vue

---



- 了解什么是Vue，能够说出Vue的基本概念

## 1.2.1 什么是Vue

---

什么是Vue呢?



## 1.2.1 什么是Vue

---



Vue（读音：/Vju:/）是一款用于构建用户界面的渐进式框架。其中，“渐进式”是指在使用Vue核心库时，可以在核心库的基础上根据实际需要逐步增加功能。

## 1.2.1 什么是Vue

---

使用Vue进行项目开发具有以下**优势**。

- ① 轻量级。Vue是一个轻量级的前端开发框架，文件体积小。
- ② Vue项目基于JavaScript语言开发，开发者不用单独学一门陌生的语言，从而**降低了学习的门槛**。
- ③ Vue在使用上比较**灵活**，开发人员可以选择使用Vue开发一个全新项目，也可以将Vue引入现有项目。

## 1.2.1 什么是Vue

---

使用Vue进行项目开发具有以下优势。

- ④ Vue通过虚拟DOM技术减少对DOM的直接操作，并通过尽可能简单的API来实现响应的数据绑定，可支持单向和双向数据绑定。
- ⑤ Vue支持组件化开发，可提高项目的开发效率和可维护性，使代码更易于复用，便于团队的协同开发。
- ⑥ Vue可以与前端开发中用到的一系列工具以及各种支持库结合使用，以实现前端工程化开发，从而提高了项目的开发效率，降低了大型项目的开发难度。

## 1.2.1 什么是Vue

---



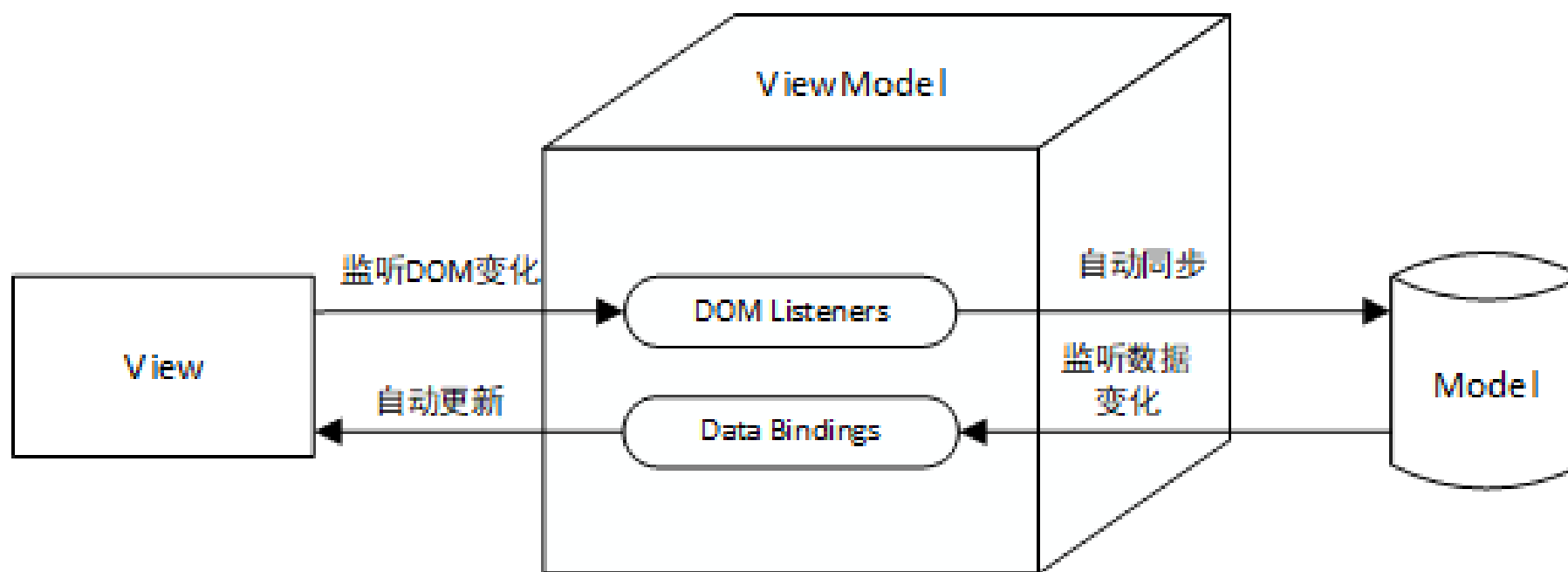
Vue是基于MVVM模式的框架。

MVVM主要包含Model（数据模型）、View（视图）和ViewModel（视图模型）。

- Model是指数据部分，负责业务数据的处理；
- View是指视图部分，即用户界面，负责视图处理；
- ViewModel用于连接视图与数据模型，负责监听Model或者View的改变。

## 1.2.1 什么是Vue

Vue的基本工作原理如下图所示。



## 1.2.1 什么是Vue

---



View和Model不能直接通信，它们需要借助ViewModel才能进行通信。ViewModel相当于一个观察者，监控着View和Model的动作，实现了View与Model的解耦。

ViewModel 包含 DOM Listeners 和 Data Bindings。

- DOM Listeners 用于监听View中DOM变化，并在DOM变化时通知Model做出相应的修改。
- Data Bindings用于监听Model中的数据变化，并在数据变化时通知View做出相应的修改。

## 1.2.2 Vue的特性

---

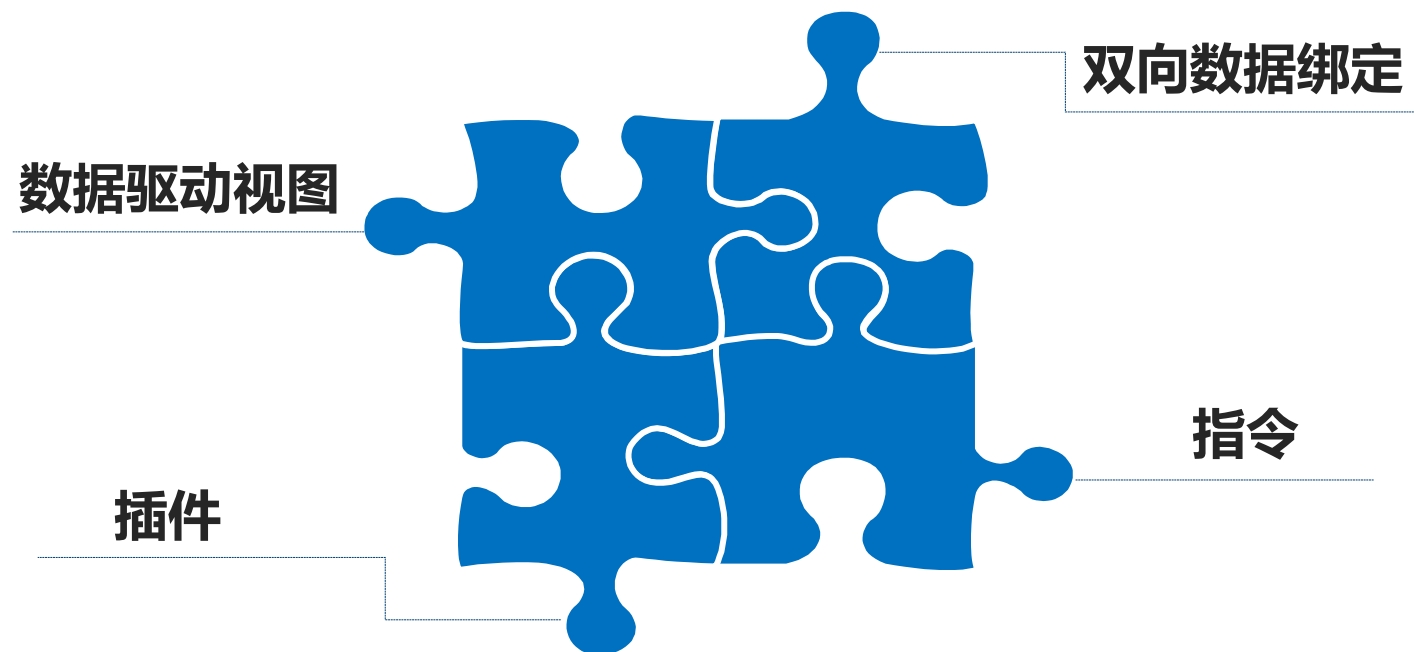


- 了解Vue的特性，能够说出Vue的4个特性

## 1.2.2 Vue的特性

---

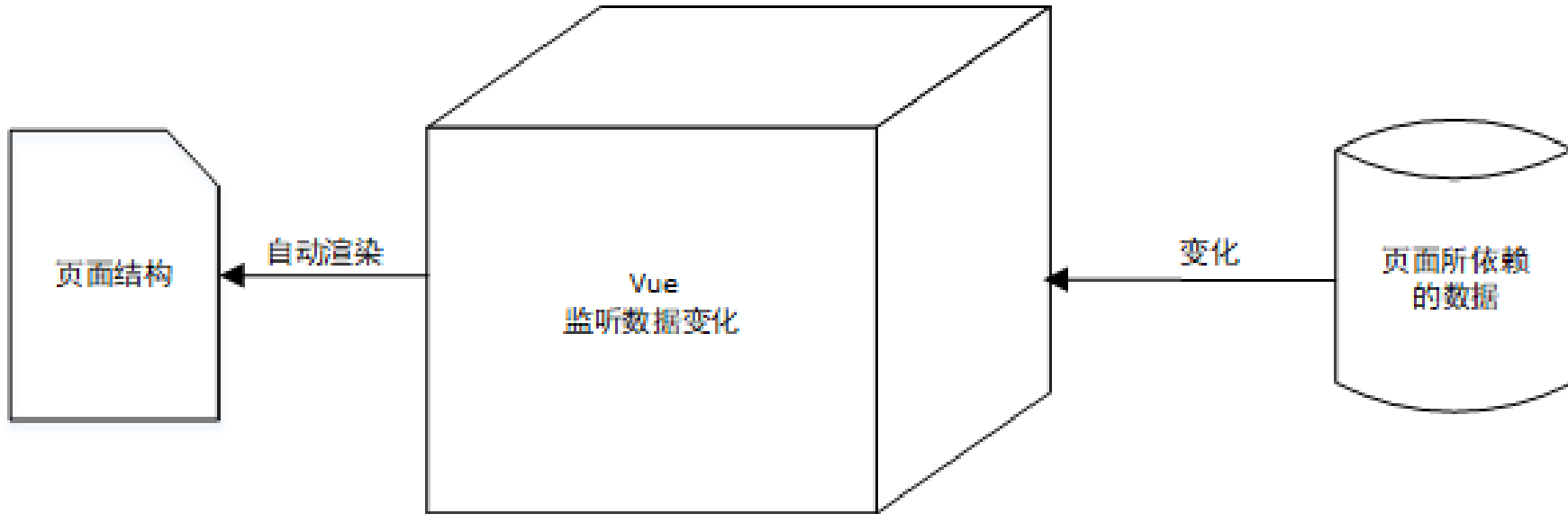
Vue的4个特性。



## 1.2.2 Vue的特性

### 1. 数据驱动视图

在使用Vue的页面中，Vue会监听数据变化，当页面数据发生变化时，Vue会自动重新渲染页面结构，如下图所示。



## 1.2.2 Vue的特性

---

### 2. 双向数据绑定

Vue实现了双向数据绑定，即当数据发生变化时，视图也会发生变化；当视图发生变化时，数据也会跟着同步变化。

例如，用户在填写表单时，双向数据绑定可以辅助开发者在无须手动操作DOM的前提下，自动同步用户填写的内容数据，从而获取表单元素最新的值。

## 1.2.2 Vue的特性

---

### 3. 指令

指令主要包括**内置指令**和**自定义指令**，内置指令是Vue本身自带的指令，而自定义指令是由用户自己定义的指令。

指令的名称以“**v-**”开头，作用于HTML中的元素。将指令绑定在元素上时，指令会给绑定的元素添加一些特殊的行为。例如，**v-bind**指令用于实现**单向数据绑定**，**v-if**指令用于实现页面**条件渲染**，**v-for**指令用于实现页面**列表渲染**等。

## 1.2.2 Vue的特性

---

### 4. 插件

Vue支持插件，通过加载插件可以实现更多的功能。

常用的插件有Vue Router（路由）、Vuex（状态管理库）、Pinia（轻量级状态管理库）等，这些插件经过简单配置就可以使用。

## 1.2.3 Vue的版本

---



- 了解Vue的版本，能够说出Vue 2和Vue 3的区别

### 1.2.3 Vue的版本

---



目前，Vue共有3个大版本，分别是Vue 1、Vue 2和Vue 3。其中，Vue 1几乎被淘汰，不建议学习与使用；Vue 2和Vue 3目前被广泛应用，并且Vue 3将会逐步替代Vue 2。

## 1.2.3 Vue的版本

---

Vue 3支持Vue 2中绝大多数的API与特性，同时Vue 3还新增了一些特有的功能，并废弃了Vue 2中的一些旧功能。

Vue 3新增的功能包括[组合式（Composition）API](#)、[多根节点组件](#)等；废弃的旧功能包括[过滤器（Filter）](#)以及`$on()`、`$off()`和`$once()`实例方法等。虽然从表面上看，Vue 3和Vue 2的使用方式没有太大的差异，但Vue 3的底层代码发生了很大变化，包括渲染、数据监听、双向绑定、生命周期等。

## 1.2.3 Vue的版本

---

Vue 3的新特性如下。

- ① 体积更小，采用按需编译的方式编译出来的文件体积比Vue 2的小。
- ② 性能提升，运行速度比Vue 2快1.5倍左右。
- ③ 具有更好的TypeScript支持。
- ④ 暴露了更底层的API，可以通过多种方式组织代码，代码使用上更加灵活。
- ⑤ 提供了更先进的组件。Vue创建了一个虚拟的Fragment节点，允许组件中有多个根节点。
- ⑥ 提供组合式API，能够更好地组合逻辑、封装逻辑、复用逻辑。

### 1.2.3 Vue的版本

---

为了提高开发效率，开发者可以在项目中添加UI组件库。UI组件库可以理解成一个可重复使用的界面设计元素的集合体，使用它可以更快速地开发用户界面。

目前，主流UI组件库都已经发布了支持Vue 3的版本，常用的UI组件库如下。

- ① **Element Plus**组件库：一款基于Vue 3的桌面端组件库。
- ② **Vant**组件库：一款开源移动端组件库，它从3.0版本开始支持Vue 3。
- ③ **Ant Design Vue**组件库：一款用于开发和服务企业级后台产品的组件库，它从2.0版本开始支持Vue 3。

**1.3**

## **Vue开发环境**

## 1.3.1 Visual Studio Code编辑器

---



- 掌握Visual Studio Code编辑器的使用方法，能够完成中文语言扩展、Volar扩展的安装，以及使用Visual Studio Code编辑器打开项目并在项目中创建一个HTML5文档

### 1.3.1 Visual Studio Code编辑器

---



**Visual Studio Code**（简称VS Code）是由微软公司推出的一款免费、开源的**代码编辑器**，一经推出便受到开发者的欢迎。对于前端开发人员来说，一个强大的编辑器可以使开发变得简单、便捷、高效。

## 1.3.1 Visual Studio Code编辑器

---

VS Code编辑器具有以下特点。

- ① 轻巧、极速，占用系统资源较少。
- ② 具备代码智能补全、语法高亮显示、自定义快捷键和代码匹配等功能。
- ③ 跨平台，可用于macOS、Windows和Linux操作系统。
- ④ 主题界面的设计比较人性化。例如，可以快速查找文件并直接进行开发，可以通过分屏显示代码，可以自定义主题颜色，以及可以快速查看已打开的项目文件和项目文件结构。
- ⑤ 提供丰富的扩展，用户可根据需要自行下载和安装扩展。

## 1.3.1 Visual Studio Code编辑器

---

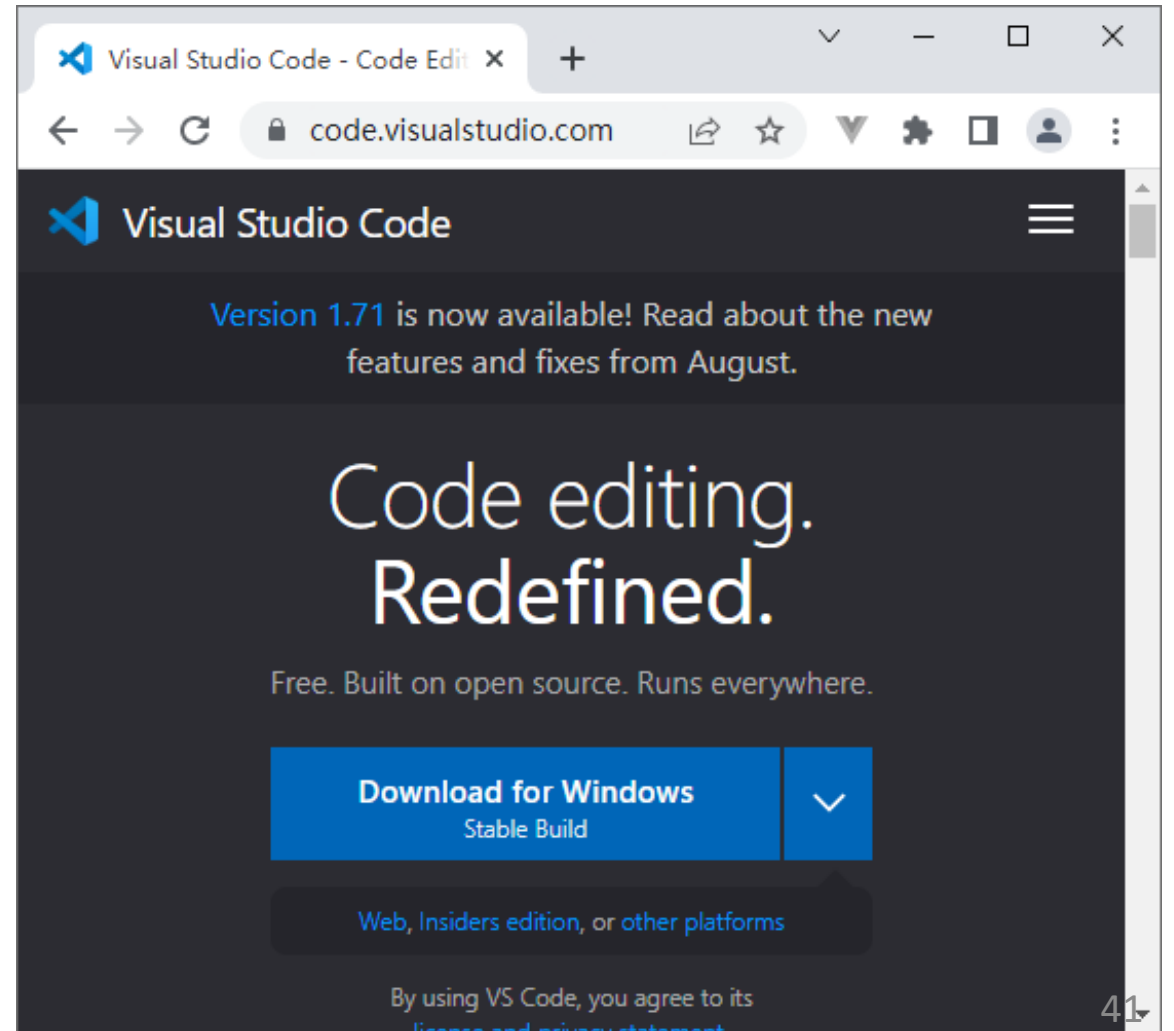


下面讲解如何下载和安装VS Code编辑器、如何安装中文语言扩展、如何安装Volar扩展，以及如何使用VS Code编辑器。

## 1.3.1 Visual Studio Code编辑器

### 1. 下载和安装VS Code编辑器

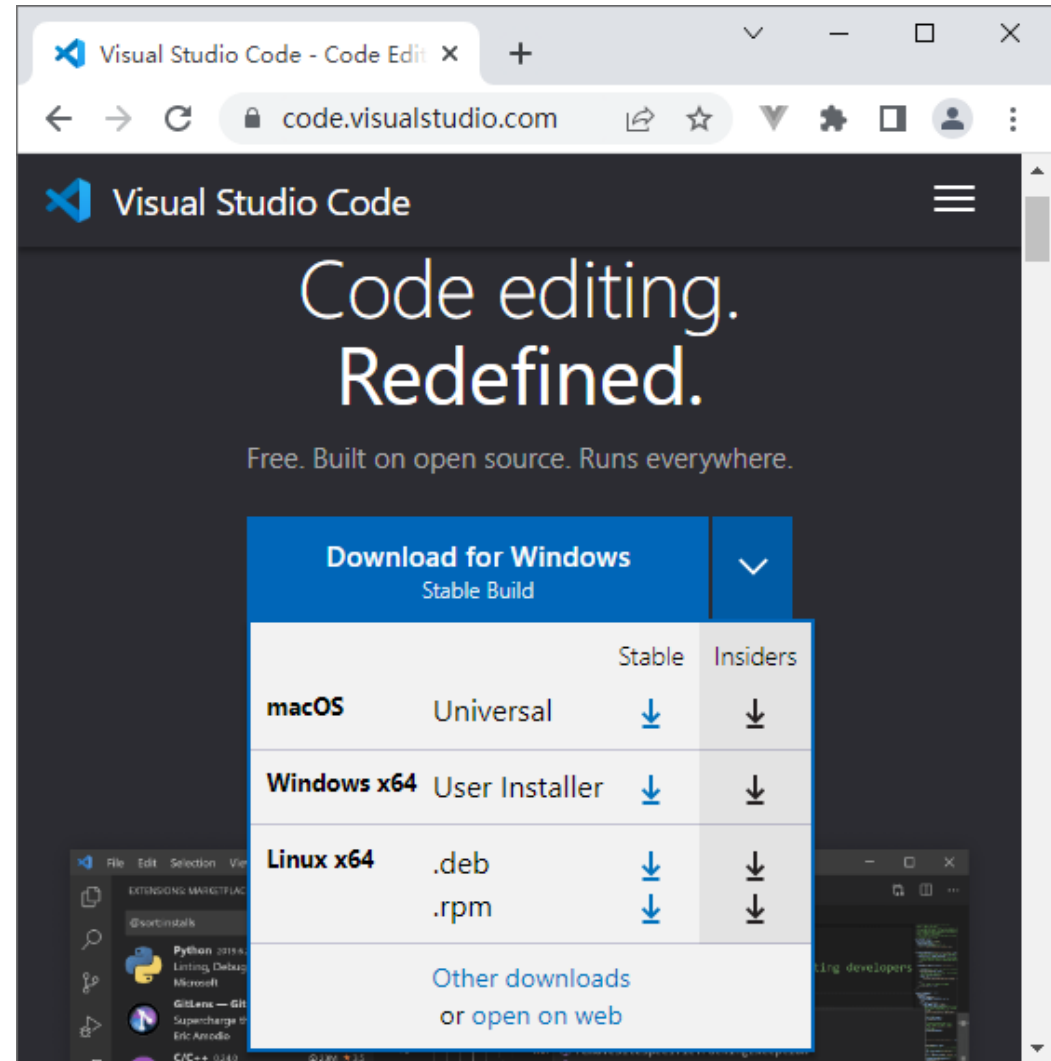
打开浏览器，登录VS Code编辑器的官方网站，如右图所示。



## 1.3.1 Visual Studio Code编辑器

### 1. 下载和安装VS Code编辑器

在VS Code编辑器的官方网站中，单击“Download for Windows”按钮，该页面会自动识别当前的操作系统并下载相应的安装包。如果需要下载其他系统的安装包，可以单击按钮右侧的小箭头“▼”打开下拉菜单，就会看到其他系统的安装包对应的下载图标，如右图所示。

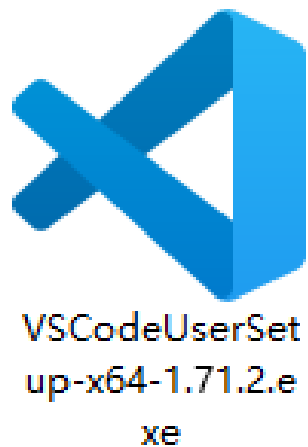


## 1.3.1 Visual Studio Code编辑器

---

### 1. 下载和安装VS Code编辑器

下载VS Code编辑器的安装包后，在下载目录中找到该安装包，如下图所示。

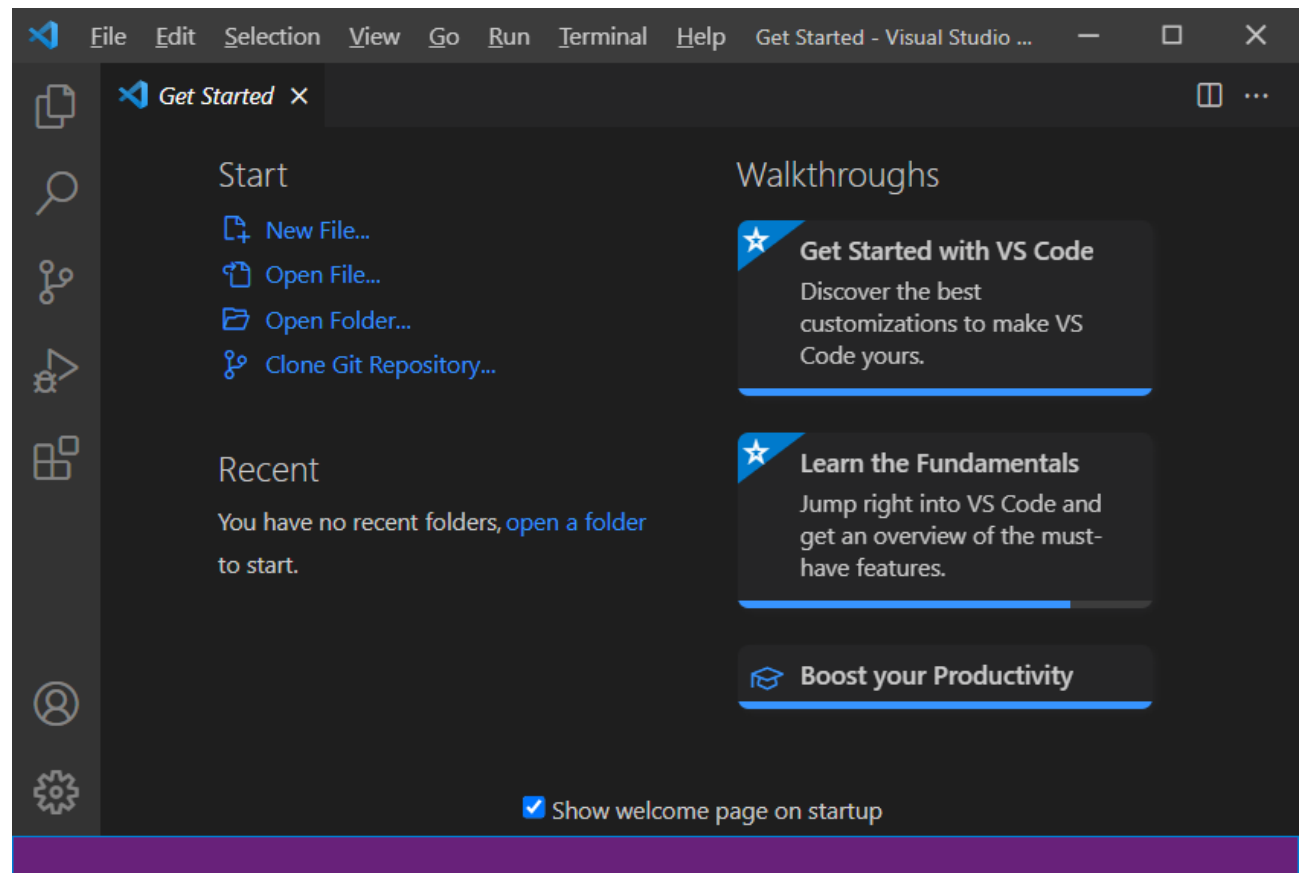


双击上图所示的图标，启动安装程序，然后按照程序的提示一步一步进行操作，直到安装完成。

## 1.3.1 Visual Studio Code编辑器

### 1. 下载和安装VS Code编辑器

将VS Code编辑器安装成功后，启动该编辑器，即可进入VS Code编辑器的初始界面，如图所示。



## 1.3.1 Visual Studio Code编辑器

### 2. 安装中文语言扩展

将VS Code编辑器安装完成后，该编辑器的默认语言是英文。如果想要切换为中文，首先单击VS Code编辑器的初始界面左侧边栏中的“”图标按钮进入扩展界面，然后在搜索框中输入关键词“chinese”找到中文语言扩展，单击“Install”按钮进行安装，如下图所示。



## 1.3.1 Visual Studio Code编辑器

### 2. 安装中文语言扩展

安装成功后，需要重新启动VS Code编辑器，中文语言扩展才可以生效。重新启动VS Code编辑器后，VS Code编辑器的中文界面如下图所示。



## 1.3.1 Visual Studio Code编辑器

### 3. 安装Volar扩展



**Volar扩展**专门用于为 “.vue” 单文件组件 (Single-File Component, SFC) 提供代码高亮显示和语法支持。该扩展仅适用于Vue 3项目，不仅提供了**语法高亮显示**、**智能提示**、**错误提示**、**格式化**等功能，而且支持多根节点组件，例如支持将一个Vue文件中的 `<template>`、`<script>`、`<style>` 拆分成3个窗口，每个窗口负责各自的功能。

## 1.3.1 Visual Studio Code编辑器

---

### 3. 安装Volar扩展

Volar扩展的安装方法与中文语言扩展的安装方法类似。如果想要安装Volar扩展，可以在扩展界面的搜索框中输入关键词“Volar”，搜索到“Vue Language Features (Volar)”扩展后进行安装。

需要注意的是，Vue 3的组件代码与Vue 2不同，使用的语法提示和高亮插件也不同。在Vue 2中不能使用Volar扩展，而是使用Vetur扩展，可以在扩展界面的搜索框中输入关键词“Vetur”，搜索到“Vetur”扩展后进行安装。

## 1.3.1 Visual Studio Code编辑器

---

### 4. 使用VS Code编辑器



在实际开发中，开发一个项目需要先创建项目文件夹，以便于保存项目中的文件。

接下来演示如何创建项目文件夹，如何使用VS Code编辑器打开项目文件夹，以及如何创建一个HTML5文档。

## 1.3.1 Visual Studio Code编辑器

### 创建项目文件夹

步骤1

在D:\vue目录下创建一个项目文件夹chapter01。

步骤2

步骤3



## 1.3.1 Visual Studio Code编辑器

### 打开文件夹

在VS Code编辑器的菜单栏中选择“文件”→“打开文件夹”命令，然后选择chapter01文件夹。打开文件夹后的界面效果如下图所示。

步骤1

步骤2

步骤3



## 1.3.1 Visual Studio Code编辑器

---

### 创建HTML5文档

#### 步骤1

单击按钮①，输入要创建的文件名称`index.html`，即可创建该文件。

#### 步骤2

此时创建的`index.html`文件是一个空白的文档，在该文档中，输入

`"html:5"`，VS Code会给出智能提示，然后按“Enter”键会自动生成一个HTML5文档结构。

#### 步骤3

## 1.3.1 Visual Studio Code编辑器

### 创建HTML5文档

步骤1

步骤2

步骤3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

## 1.3.2 Node.js环境

---



- 掌握Node.js环境的搭建，能够独立完成Node.js的下载和安装

## 1.3.2 Node.js环境

---



Node.js是一个基于V8引擎的JavaScript运行环境，提供了一些功能性的API，例如文件操作API、网络通信API等。

## 1.3.2 Node.js环境

---

打开[Node.js官网](#)，找到Node.js下载地址，如下图所示。



Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine.

Download for Windows (x64)

**16.17.0 LTS**

Recommended For Most Users

**18.9.0 Current**

Latest Features

[Other Downloads](#) | [Changelog](#) | [API Docs](#)   [Other Downloads](#) | [Changelog](#) | [API Docs](#)

Or have a look at the [Long Term Support \(LTS\) schedule](#)

## 1.3.2 Node.js环境

---

从[Node.js官方网站](#)可以看出，Node.js有两个版本，分别是16.17.0 LTS和18.9.0 Current。

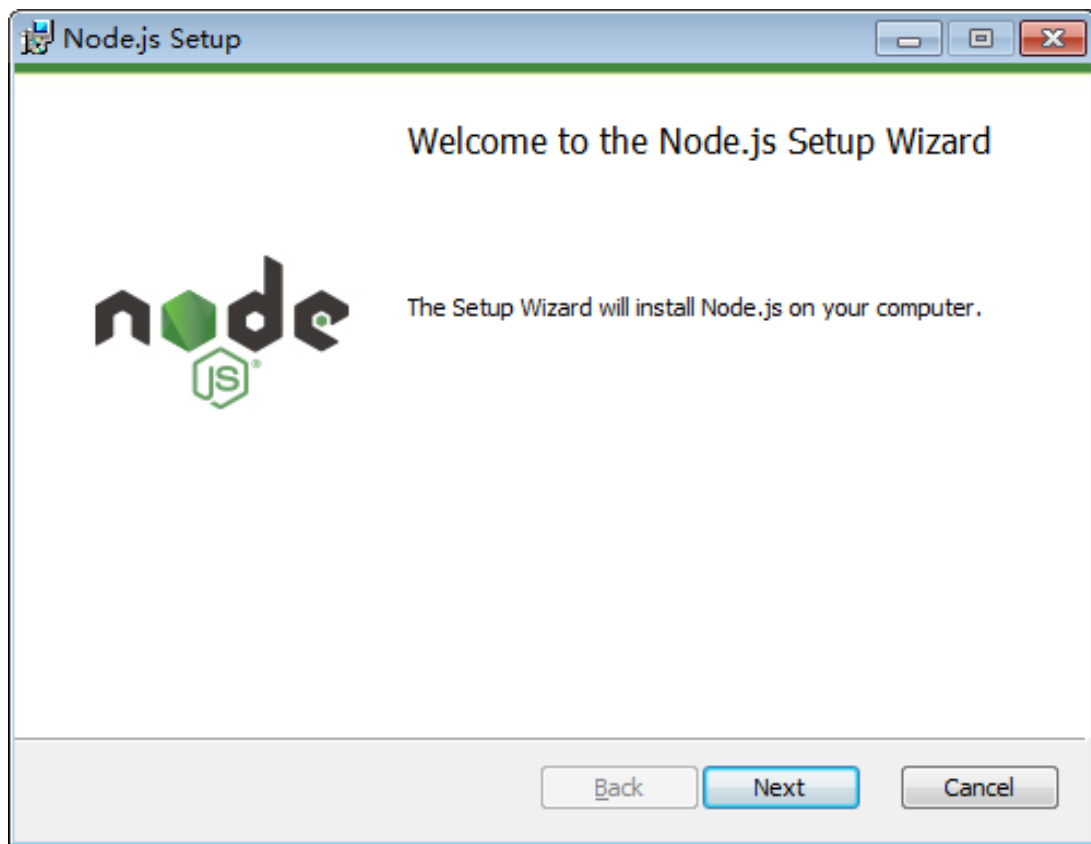
其中，**LTS**（Long Term Support）表示提供[长期支持的版本](#)，只进行Bug修复且版本稳定，因此有很多用户在使用；**Current**表示[当前发布的新版本](#)，增加了一些新特性，有利于进行新技术的开发使用。

这里选择下载16.17.0 LTS版本。下载完成后会得到一个名称为node-v16.17.0-x64.msi的安装包文件。

## 1.3.2 Node.js环境

---

双击node-v16.17.0-x64.msi安装包图标，会弹出[安装向导窗口](#)，如下图所示。安装过程全部使用默认值。



## 1.3.2 Node.js环境

---



安装完成后，测试一下Node.js是否安装成功。

## 1.3.2 Node.js环境

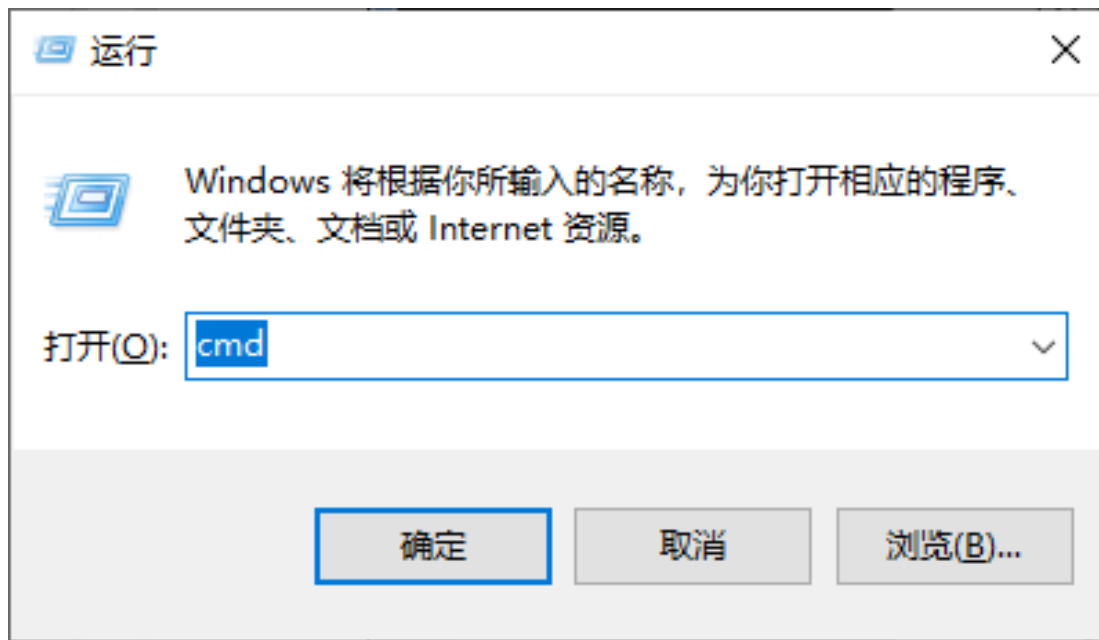
### 打开“运行”对话框

步骤1

按“Windows+R”组合键，打开“运行”对话框，输入“cmd”。在“运行”对话框中输入“cmd”后的效果如下图所示。

步骤2

步骤3



## 1.3.2 Node.js环境

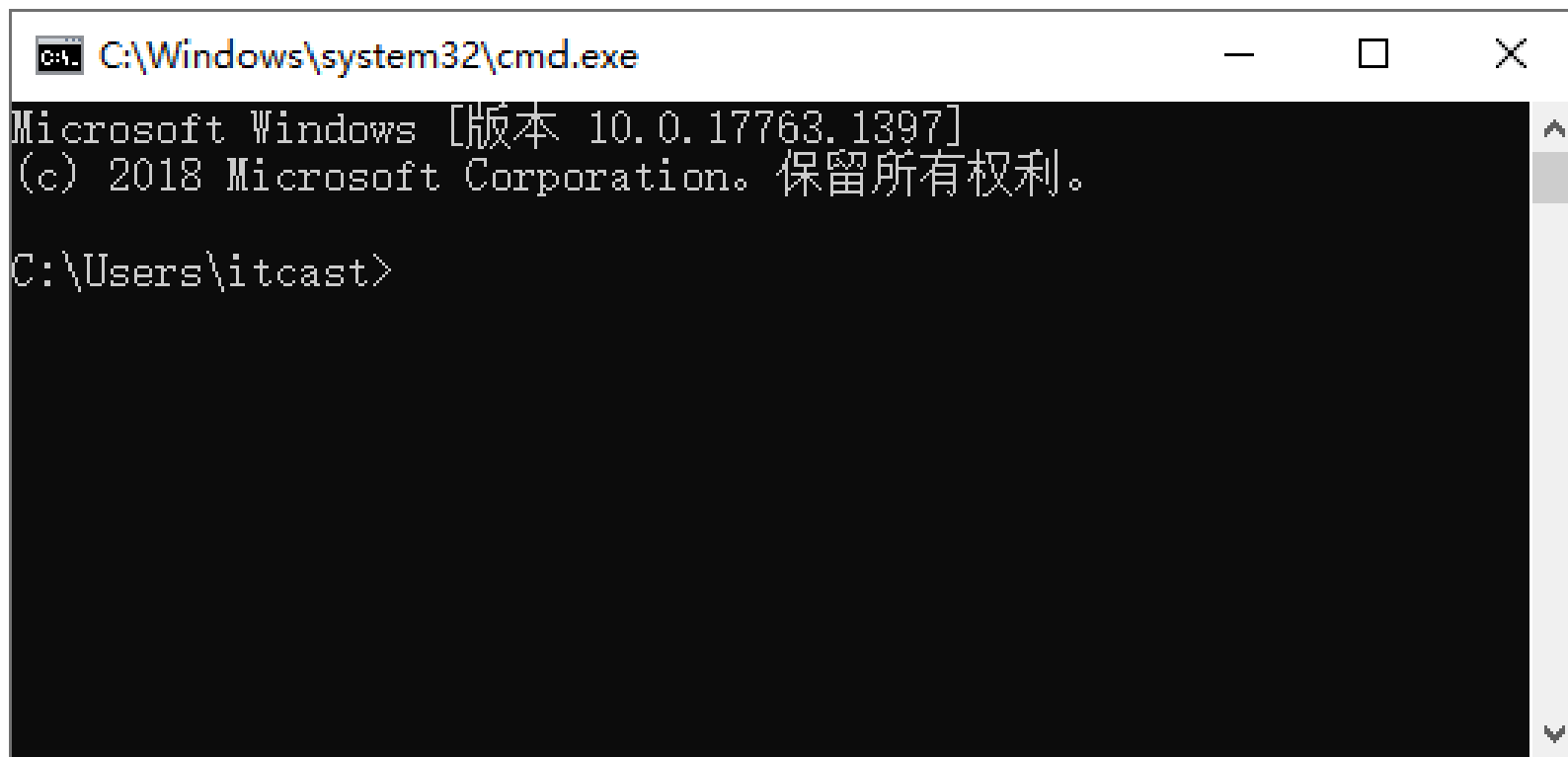
### 打开命令提示符

单击“确定”按钮或者直接按“Enter”键，会打开命令提示符，如下图所示。

步骤1

步骤2

步骤3



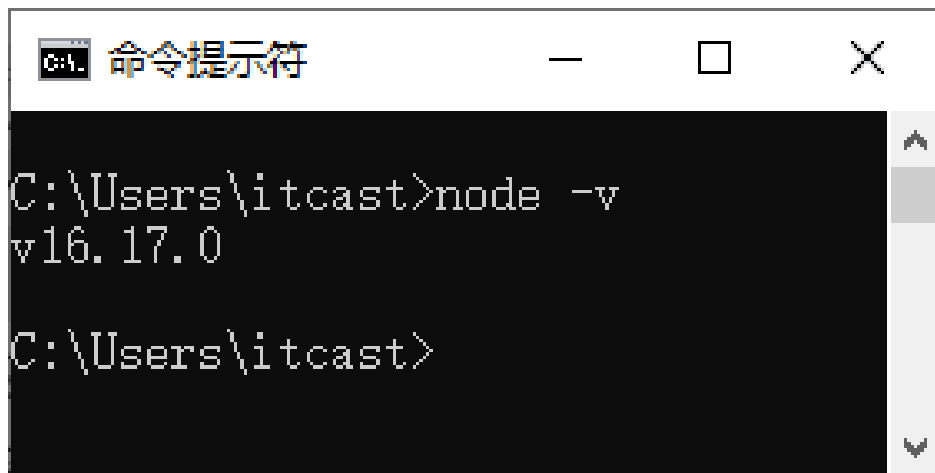
## 1.3.2 Node.js环境

### 查看Node.js版本

步骤1

在命令提示符中，输入命令 “`node -v`”，其中v是version的简写，表示版本。命令输入完成后，按 “Enter” 键，查看当前安装的Node.js版本，如下图所示。

步骤2



```
C:\Users\itcast>node -v
v16.17.0
C:\Users\itcast>
```

步骤3

若想要退出命令提示符，可以输入 “`exit`” 并按 “Enter” 键，或者单击右上角的 “x” 关闭按钮退出。

### 1.3.3 常见的包管理工具

---



- 掌握常见的包管理工具的使用方法，能够应用npm和yarn相关命令下载、升级、卸载包

### 1.3.3 常见的包管理工具

---

什么是包呢？



### 1.3.3 常见的包管理工具

---



在Vue项目开发中，经常需要通过各种第三方的包（package）来扩展项目的功能。“包”可以理解为将一系列模块化的代码打包起来，形成一个包，以便于使用。项目中所用到的包称为项目的依赖（dependency）。

### 1.3.3 常见的包管理工具

---

什么是包管理工具呢？



### 1.3.3 常见的包管理工具

---



为了方便管理第三方的包，需要用到包管理工具。包管理工具可以让开发人员轻松地下载、升级、卸载包。假设在项目开发时，没有包管理工具，若想使用第三方包，则每次都需要下载、解压后才可以使⽤，非常烦琐。而使⽤包管理工具，只需通过一条命令即可下载并安装第三方包，非常方便，而且还可以指定下载的版本等。

### 1.3.3 常见的包管理工具

---

#### 常见的包管理工具



#### npm

npm是Node.js默认的包管理工具，它可以安装、共享、分发代码，还可以管理项目的依赖关系。



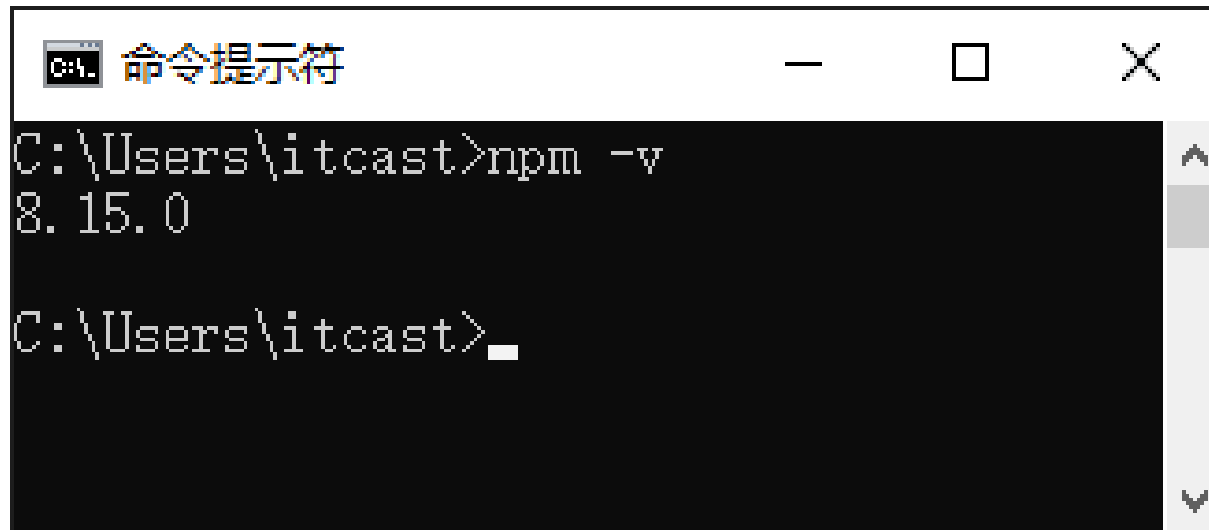
#### yarn

yarn是Node.js的包管理工具，它是一个高效、安全和可靠的包管理工具，yarn能够提高包的安装效率，节约安装时间。

### 1.3.3 常见的包管理工具

#### 1. npm

在安装Node.js时会[自动安装相应版本的npm](#)，不需要单独安装，使用“`npm -v`”命令可以查看npm的版本，如下图所示。



```
C:\Users\itcast>npm -v
8.15.0
C:\Users\itcast>_
```

### 1.3.3 常见的包管理工具

---

使用npm包管理工具可以解决以下场景的需求。

- ① 从npm服务器下载别人编写的第三方包到本地使用。
- ② 从npm服务器下载并安装别人编写的命令行程序到本地来使用。
- ③ 将自己编写的包或命令行程序上传到npm服务器供别人使用。

### 1.3.3 常见的包管理工具

---

npm提供了快速操作包的命令，只需要执行简单的命令就可以很方便地对第三方包进行管理。

npm中常用的命令如下。

- `npm install 包名`：可简写为“`npm i 包名`”，用于为项目安装指定名称的包。如果加上-g选项，则会把包安装为全局包，否则只安装到本项目中。如果省略包名，则npm会根据当前目录下的package.json文件中保存的依赖信息为项目安装所有的包。
- `npm uninstall 包名`：用于卸载指定名称的包。
- `npm update 包名`：用于更新指定名称的包。

### 1.3.3 常见的包管理工具

---

在下载npm安装包时，下载速度可能会比较慢，这是因为提供包的服务器在国外。为了**加快包的下载速度**，建议将下载源切换成**国内镜像服务器**。为**npm设置镜像地址**的具体命令如下。

```
npm config set registry https://registry.npmmirror.com
```

为了验证镜像地址是否设置成功，可以通过如下命令进行验证。

```
npm config get registry
```

执行上述命令后，若输出了设置的镜像地址，则表示设置成功。

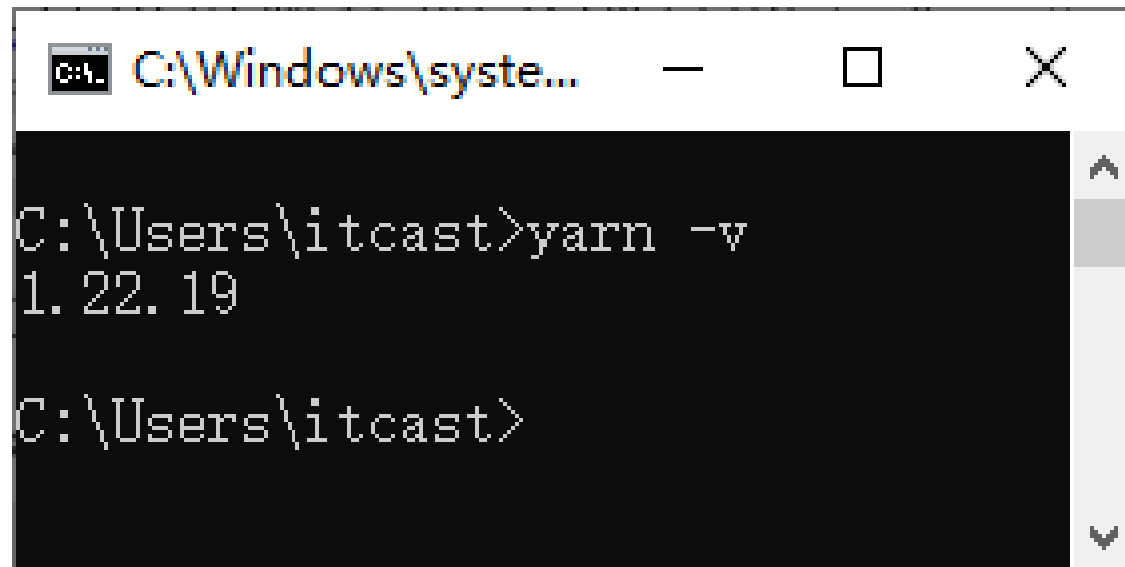
### 1.3.3 常见的包管理工具

#### 2. yarn

在使用yarn之前，需要先确保计算机中已经安装了Node.js环境，再使用npm命令安装yarn，安装命令如下。

```
npm install yarn -g
```

为了验证yarn是否安装成功，可以通过“yarn -v”命令查看yarn的版本信息，如下图所示。



```
C:\Windows\system32\cmd.exe
C:\Users\itcast>yarn -v
1.22.19
C:\Users\itcast>
```

### 1.3.3 常见的包管理工具

---

为了提高下载yarn安装包的速度，也可以将yarn的下载源切换成国内镜像服务器。为yarn设置镜像地址的具体命令如下。

```
yarn config set registry https://registry.npmmirror.com
```

为了验证镜像地址是否设置成功，可以通过如下命令进行验证。

```
yarn config get registry
```

执行上述命令后，若输出了设置的镜像地址，则表示设置成功。

### 1.3.3 常见的包管理工具

---

下面列举yarn中一些常用的命令。

- `yarn install`: 可简写为yarn, 用于为项目安装所有包。如果提供了-g选项, 则会把包安装为全局包, 否则只安装到本项目中。
- `yarn remove 包名`: 用于卸载指定名称的包。
- `yarn up 包名`: 用于更新指定名称的包。
- `yarn add 包名`: 用于添加指定名称的包。

### 1.3.3 常见的包管理工具

#### yarn与npm包管理工具的区别

- ① 使用npm安装同一个包时，每次安装都需要重新下载，而yarn会缓存每个下载过的包，再次使用时无须重复下载。
- ② npm按照队列安装每个包，也就是说，必须要等到当前包安装完成后，才能继续安装后面的包，而yarn可以利用并行下载的方式提高资源利用率，安装速度更快。
- ③ npm的输出信息比较冗长，在执行npm install命令时，命令提示符里会输出所有被安装的包的信息。相比之下，yarn的输出信息比较简洁，只输出必要的信息，同时也提供了一些命令供开发者查询额外的安装信息。

# 1.4

## 使用Vite创建Vue 3项目

## 1.4.1 什么是Vite

---



- 了解什么是Vite，能够说出Vite的基本概念

## 1.4.1 什么是Vite

---

什么是Vite?



## 1.4.1 什么是Vite

---



**Vite**（读音：/vit/）是一个轻量级、运行速度快的前端构建工具，它支持**模块热替换**（Hot Module Replacement, HMR），可以即时、准确地更新模块，当代码修改时无须重新加载页面或清除应用程序状态。Vite默认安装的插件比较少，随着开发过程中依赖的增多，需要额外进行配置。

## 1.4.1 什么是Vite

---

在Vue 3出现前，Vue 2一般使用Vue CLI创建。Vue CLI基于Webpack构建并配置项目，在项目启动时，Webpack需要从入口文件索引整个项目的文件，编译成一个或多个单独的js文件。虽然Webpack对代码进行了拆分，但是仍可能一次生成所有路由下的编译后的文件，导致服务启动时间随着项目的复杂度增加而呈指数式的增长。而Vite改进了这一点，在项目启动时，Vite会对模块代码进行按需加载，启动速度更快。因此，当使用Vue3 开发新项目时，推荐使用Vite进行创建。

## 1.4.1 什么是Vite

---

Vite不需要以命令的方式安装和卸载，只要安装了npm或yarn，就可使用Vite的相关命令创建项目。Vite会作为项目的开发依赖保存在项目的node\_modules目录中。

需要注意的是，Node.js必须为14.18及以上版本时才可以使用Vite，并且Vite中的部分模板可能需要更高的Node.js版本才能正常运行。另外，Node.js 的14、16等更新版本不再支持Windows 7操作系统，推荐使用Windows 10等新版操作系统。

## 1.4.2 创建Vue 3项目

---



- 掌握Vue 3项目的创建方法，能够使用Vite创建Vue 3项目

## 1.4.2 创建Vue 3项目

---



Vite提供了两种创建项目的命令。

- 手动创建项目的命令
- 通过模板自动创建项目的命令

## 1.4.2 创建Vue 3项目

---

### 1. 手动创建项目的命令

使用npm或yarn包管理工具都可以搭配Vite手动创建项目，具体命令如下。

```
# 使用npm create命令创建项目
npm create vite@latest
# 使用yarn create命令创建项目
yarn create vite
```

上述命令展示了两种包管理工具用于创建Vite项目，在使用时任选其一即可。npm create和yarn create命令后跟一个vite包名，表示初始化Vite。vite@latest表示在npm中安装最新版本的Vite。

## 1.4.2 创建Vue 3项目

### 手动创建Vue项目

#### 步骤1

打开命令提示符，切换到D:\vue\chapter01目录，执行如下命令。

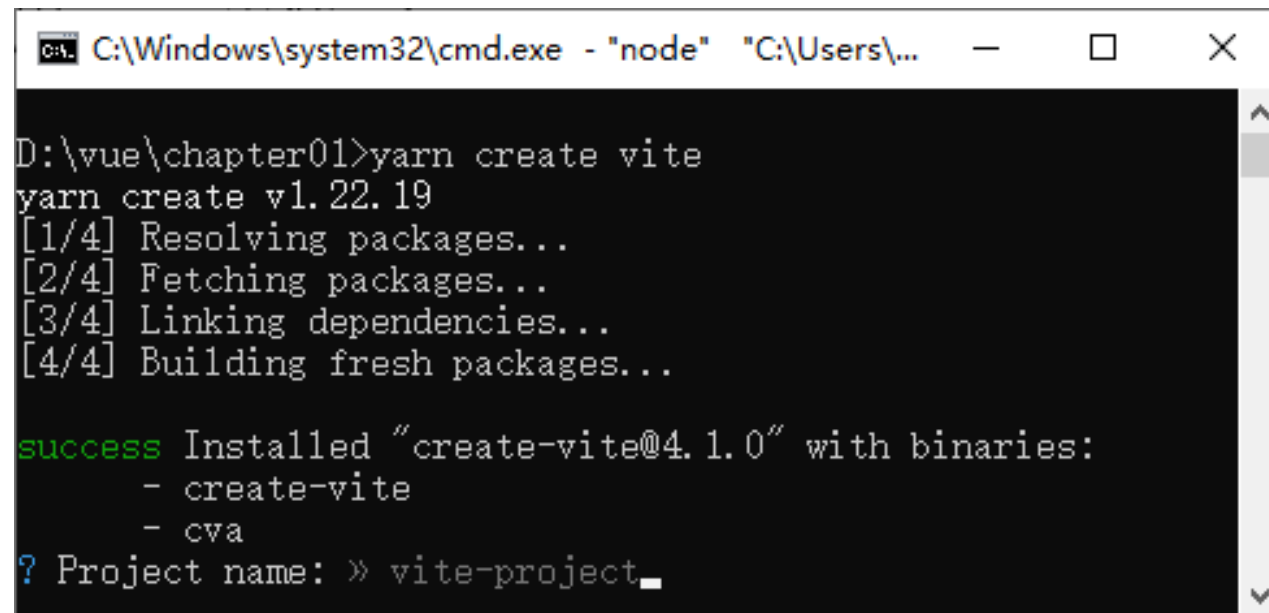
```
yarn create vite
```

#### 步骤2

执行上述命令后，Vite会提示填写项目名称，如下图所示。

#### 步骤3

#### 步骤4



```
C:\Windows\system32\cmd.exe - "node" "C:\Users\...\
D:\vue\chapter01>yarn create vite
yarn create v1.22.19
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...

success Installed "create-vite@4.1.0" with binaries:
  - create-vite
  - cva
? Project name: » vite-project_
```

## 1.4.2 创建Vue 3项目

### 手动创建Vue项目

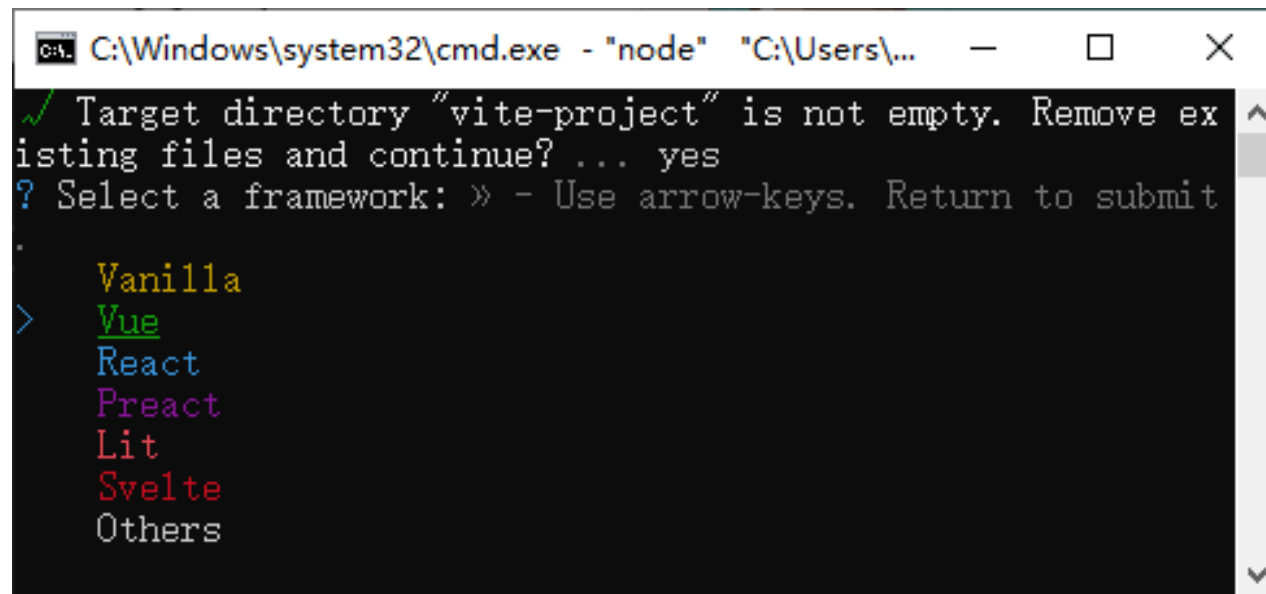
步骤1

步骤2

步骤3

步骤4

使用vite-project作为项目名称后，Vite会提示选择创建项目所使用的框架，如下图所示。



```
C:\Windows\system32\cmd.exe - "node" "C:\Users\...\vite-project"
✓ Target directory "vite-project" is not empty. Remove existing files and continue? ... yes
? Select a framework: » - Use arrow-keys. Return to submit
  Vanilla
>  Vue
  React
  Preact
  Lit
  Svelte
  Others
```

## 1.4.2 创建Vue 3项目

### 手动创建Vue项目

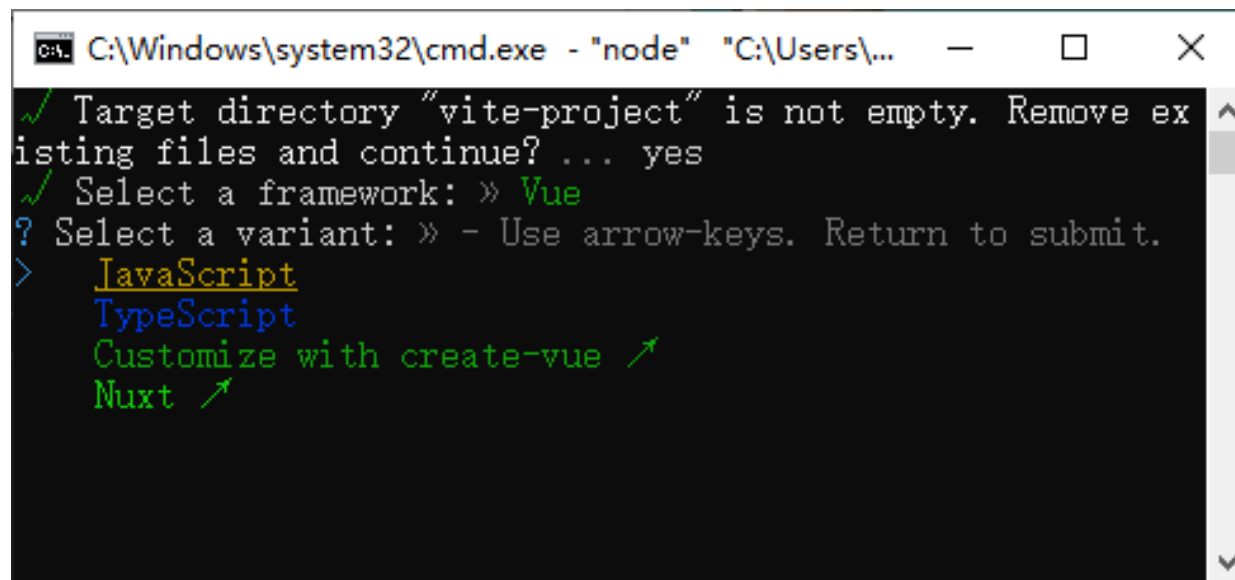
步骤1

步骤2

步骤3

步骤4

选择好框架后，Vite会提示选择一个变体，如下图所示。



```
C:\Windows\system32\cmd.exe - "node" "C:\Users\...  
✓ Target directory "vite-project" is not empty. Remove existing files and continue? ... yes  
✓ Select a framework: » Vue  
? Select a variant: » - Use arrow-keys. Return to submit.  
> JavaScript  
TypeScript  
Customize with create-vue ↗  
Nuxt ↗
```

## 1.4.2 创建Vue 3项目

### 手动创建Vue项目

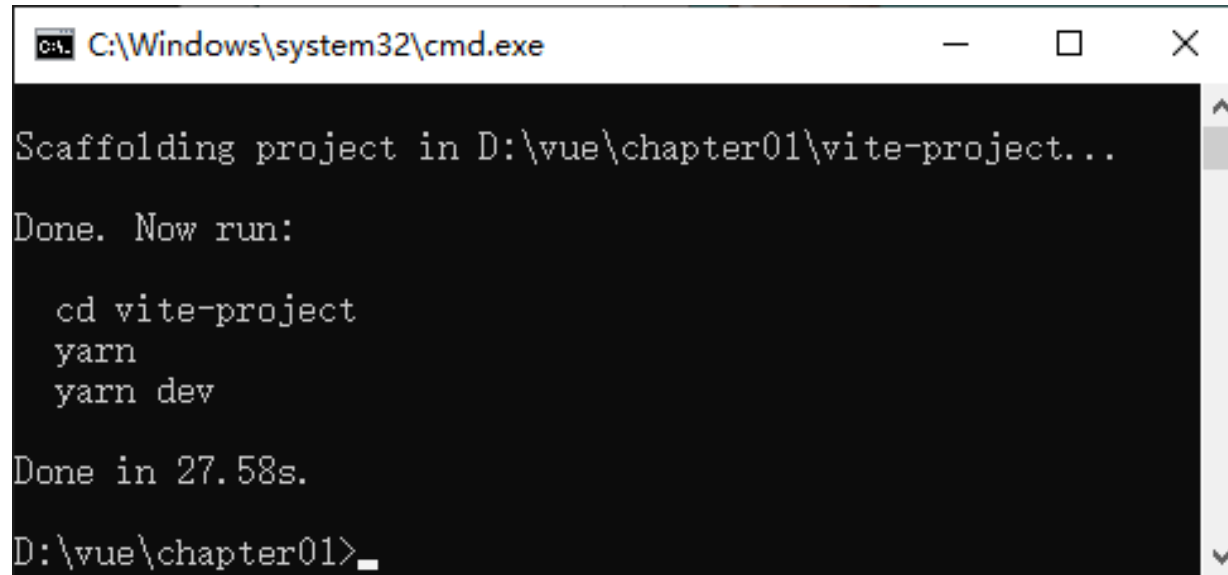
选择好变体后，Vite会提示项目创建完成，如下图所示。

步骤1

步骤2

步骤3

步骤4



```
C:\Windows\system32\cmd.exe

Scaffolding project in D:\vue\chapter01\vite-project...

Done. Now run:

  cd vite-project
  yarn
  yarn dev

Done in 27.58s.

D:\vue\chapter01>
```

## 1.4.2 创建Vue 3项目

---

项目创建完成后的提示命令，具体命令解释如下。

|                              |             |
|------------------------------|-------------|
| <code>cd vite-project</code> | # 切换到项目目录   |
| <code>yarn</code>            | # 安装项目的全部依赖 |
| <code>yarn dev</code>        | # 启动服务      |

上述命令中，`yarn dev`命令是Vue3项目中`package.json`文件里面`scripts`节点配置的命令。除了`yarn dev`命令外，还有2个常用命令`yarn build`和`yarn preview`，它们分别表示构建生产环境项目和构建本地预览环境项目。这3个命令实际上都是别名，是为了便于开发人员记忆。当执行这3个命令时，`yarn`会读取当前项目的`package.json`文件中的命令配置，找到真正的命令并执行。

## 1.4.2 创建Vue 3项目

---

Vue 3项目的package.json文件中的命令配置如下。

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview"  
},
```

上述配置中，**dev**是**vite**的别名，**build**是**vite build**的别名，**preview**是**vite preview**的别名。也就是说，当执行yarn dev时，实际执行的命令是yarn vite。别名可以自定义，如果修改了别名，在执行命令时需要使用修改后的别名。

## 1.4.2 创建Vue 3项目

---

项目启动后，会默认开启一个本地服务，具体如下。

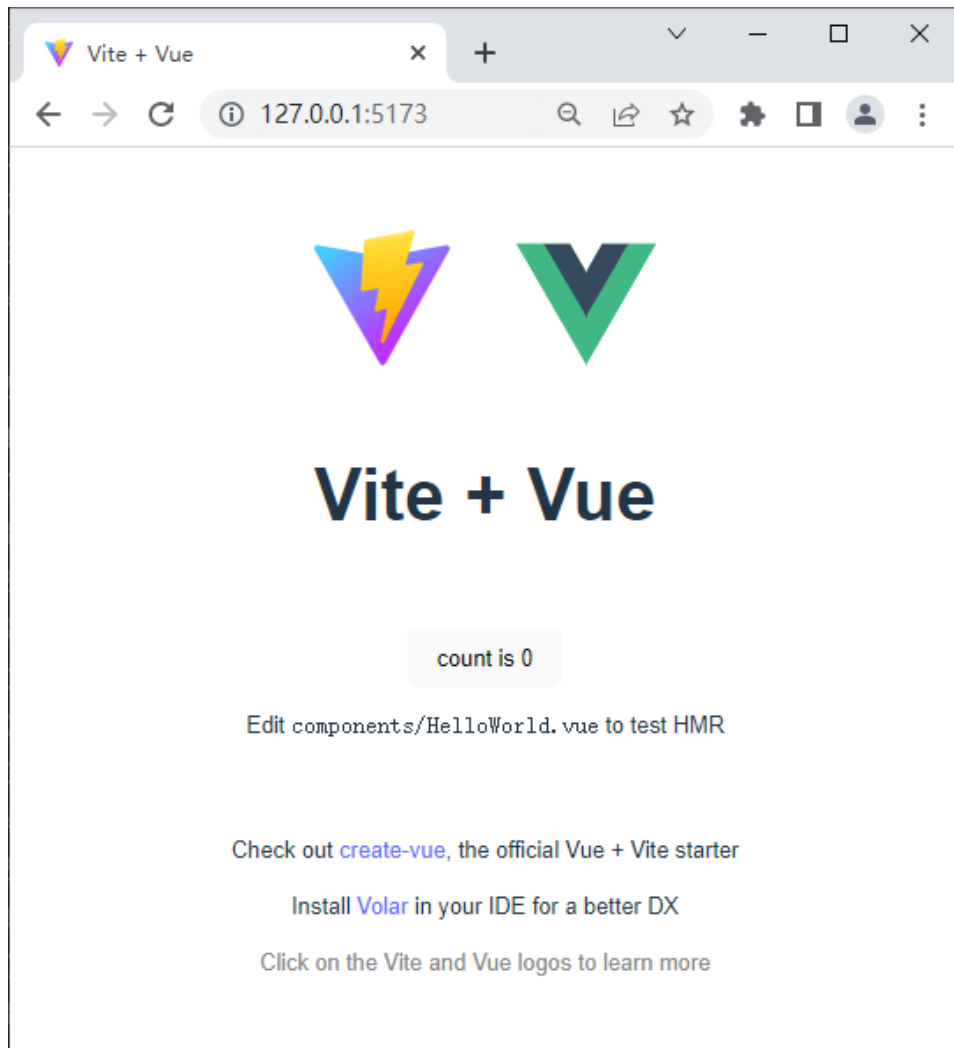
```
VITE v4.1.4 ready in 441 ms
```

```
→ Local: http://127.0.0.1:5173/
```

## 1.4.2 创建Vue 3项目

---

在浏览器中打开<http://127.0.0.1:5173/>，如下图所示。



## 1.4.2 创建Vue 3项目

### 2. 通过模板自动创建项目的命令

通过模板自动创建项目的方式相对简单，它通过附加的命令行选项直接指定项目名称和模板，省去了填写项目名称、选择框架和变体等环节。Vite提供了许多模板预设，可以创建 **Vite+React+TS**、**Vite+Vue**、**Vite+Svelte**等类型的项目。通过附加的命令行选项直接指定项目名称和模板的基本语法格式如下。

```
# 使用npm 6或更低版本创建项目
npm create vite@latest <项目名称> --template <模板名称>

# 使用npm 7或更高版本创建项目
npm create vite@latest <项目名称> -- --template <模板名称>

# 使用yarn create命令创建项目
yarn create vite <项目名称> --template <模板名称>
```

## 1.4.2 创建Vue 3项目

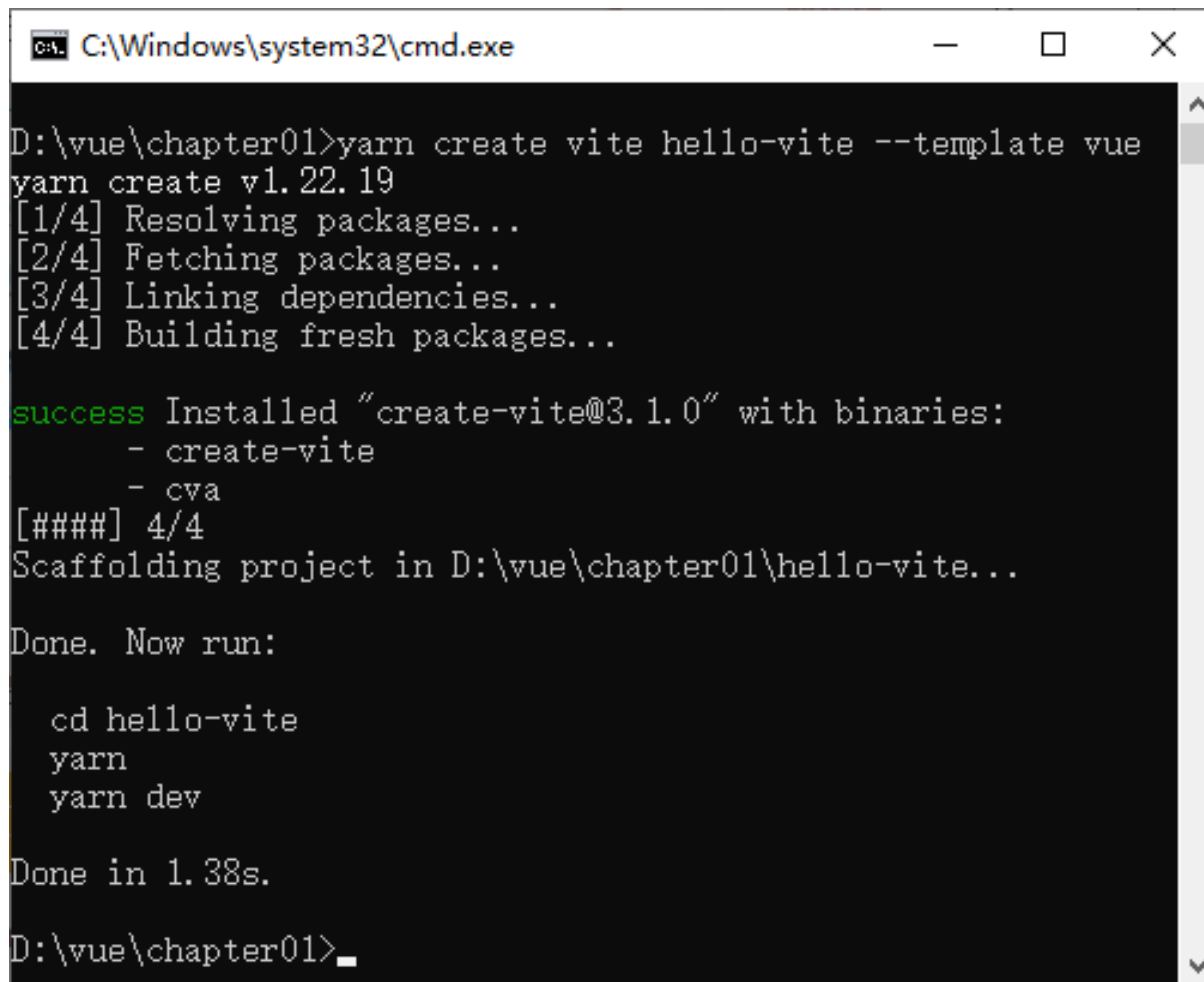
---

打开命令提示符，切换到D:\vue\chapter01目录，使用yarn创建一个基于Vite+Vue模板且项目名称为hello-vite的项目。

```
yarn create vite hello-vite --template vue
```

## 1.4.2 创建Vue 3项目

执行上述命令后，[hello-vite项目](#)创建完成的效果如下图所示。



```
C:\Windows\system32\cmd.exe

D:\vue\chapter01>yarn create vite hello-vite --template vue
yarn create v1.22.19
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...

success Installed "create-vite@3.1.0" with binaries:
  - create-vite
  - cva
[####] 4/4
Scaffolding project in D:\vue\chapter01\hello-vite...

Done. Now run:

  cd hello-vite
  yarn
  yarn dev

Done in 1.38s.
D:\vue\chapter01>_
```

## 1.4.2 创建Vue 3项目

---

在项目创建完成后，给出了一些提示命令，需要执行这些命令才能完成项目启动，具体命令如下。

|                            |             |
|----------------------------|-------------|
| <code>cd hello-vite</code> | # 切换到项目目录   |
| <code>yarn</code>          | # 安装项目的全部依赖 |
| <code>yarn dev</code>      | # 启动服务      |

项目启动后，会默认开启一个本地服务，具体如下。

```
VITE v4.1.4 ready in 441 ms
➔ Local: http://127.0.0.1:5173/
```

在浏览器中打开<http://127.0.0.1:5173/>，页面效果与手动创建Vue 3项目的页面效果相同。

### 1.4.3 Vue 3项目的目录结构

---

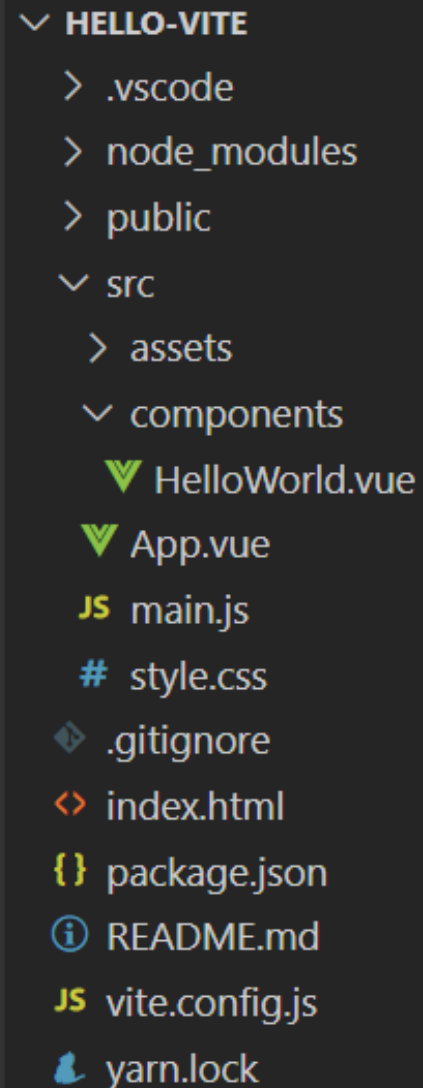


- 熟悉Vue 3项目的目录结构，能够归纳常用的目录和文件的作用

### 1.4.3 Vue 3项目的目录结构

---

Vue 3项目创建成功后，使用VS Code编辑器打开项目目录，可以看到一个默认生成的[项目目录结构](#)，如右图所示。



```

✓ HELLO-VITE
  > .vscode
  > node_modules
  > public
  ✓ src
    > assets
    ✓ components
      ▼ HelloWorld.vue
      ▼ App.vue
    JS main.js
    # style.css
    .gitignore
    <> index.html
    {} package.json
    ⓘ README.md
    JS vite.config.js
    yarn.lock

```

The image shows a file explorer view of a project named 'HELLO-VITE'. The project contains several directories and files: '.vscode', 'node\_modules', 'public', 'src', 'assets', 'components' (containing 'HelloWorld.vue' and 'App.vue'), 'main.js', 'style.css', '.gitignore', 'index.html', 'package.json', 'README.md', 'vite.config.js', and 'yarn.lock'. The 'src' directory is expanded, showing its contents.

### 1.4.3 Vue 3项目的目录结构

---

下面简要介绍Vue 3项目的目录结构中各个目录和文件的作用，具体如下。

- `.vscode`: 存放VS Code编辑器的相关配置。
- `node_modules`: 存放项目的各种依赖和安装的插件。
- `public`: 存放不可编译的静态资源文件，当进行项目构建时，该目录下的文件会被复制到dist目录，该目录下的文件需要使用绝对路径访问。
- `src`: 源代码目录，保存开发人员编写的项目源代码。
- `src\assets`: 存放可编译的静态资源文件，例如图片、样式文件等。该目录下的文件需要使用相对路径访问。

### 1.4.3 Vue 3项目的目录结构

---

- `src\components`: 存放单文件组件, 即.vue文件。
- `src\components\HelloWorld.vue`: 一个名称为HelloWorld的单文件组件。
- `src\App.vue`: 项目的根组件。
- `src\main.js`: 项目的入口文件, 用于创建Vue应用实例。
- `src\style.css`: 项目的全局样式表文件。
- `.gitignore`: 向Git仓库上传代码时需要忽略的文件列表。
- `index.html`: 默认的主渲染页面文件, 同时也是页面的入口文件。

### 1.4.3 Vue 3项目的目录结构

---

- `package.json`: 包配置文件。
- `README.md`: 项目使用说明文件。
- `vite.config.js`: 存放Vite的相关配置。
- `yarn.lock`: 存储每一个依赖项的安装版本, 在使用yarn安装、升级、卸载依赖时, 会自动更新yarn.lock文件。

此外, 当执行了`yarn build`命令后, 在项目目录中会出现`dist`目录, 该目录中保存的是项目构建后的文件。

## 1.4.4 Vue 3项目的运行过程

---



- 了解Vue 3项目的运行过程，能够说出Vue 3项目的运行过程

## 1.4.4 Vue 3项目的运行过程

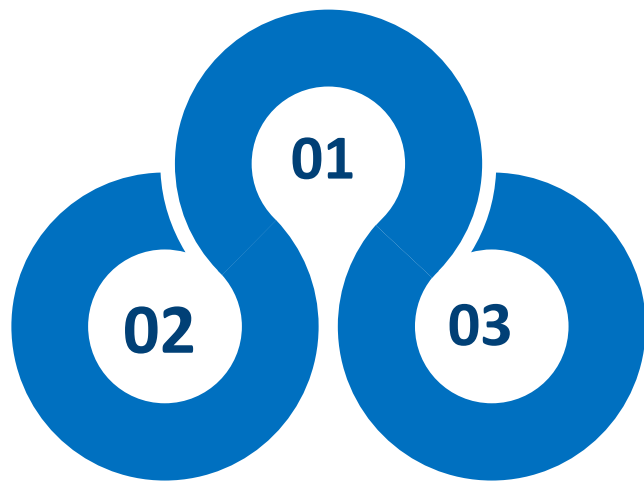
---



使用Vite构建Vue 3项目后，当执行`yarn dev`命令启动服务时，项目就会运行起来，该项目会通过`src/main.js`文件将`src/App.vue`组件渲染到`index.html`文件的指定区域。

## 1.4.4 Vue 3项目的运行过程

---



1

### src\App.vue文件

Vue 3项目是由各种组件组成的，`src\App.vue`文件是项目的根组件。在根组件中可以引用其他组件，从而显示其他组件的内容。

2

### index.html文件

`index.html`文件是页面的入口文件，该文件中预留了用于被 `src/main.js` 文件中的Vue应用实例所控制的区域。

3

### src\main.js文件

`src/main.js`文件是项目的入口文件，该文件创建了Vue应用实例。Vue应用实例是Vue项目工作的基础，每个Vue项目都是从创建Vue应用实例开始的。

### 本章小结

本章首先介绍了前端技术的发展，接着对Vue的相关概念进行了简要介绍；然后讲解了Vue开发环境，包括Visual Studio Code编辑器、Node.js环境以及常见的包管理工具；最后分别讲解了如何使用Vite创建Vue 3项目。通过本章的学习，读者应对Vue有一个整体的认识，能够创建Vue 3项目。